

4. Build the application.

If you use the Profiler tool, make sure the build configuration is Target-Debug.

5. Start the run by doing one of the following:

- a) In the Project Explorer, right-click the project and select **Run As > bada Target Application**.
- b) In the bada IDE menu, go to **Run > Run As > bada Target Application**.

The executable binary, resource files, and data files are downloaded before starting the run, and the application is installed automatically while it runs.

While the application is running, the IDE's Output view shows the log, debug, and exception messages from the methods defined in the Log Macros.

6. While running the application, you can speed up repetitive tasks, such as login sequences, by using the UI Sequencer tool.
7. To terminate the run, do one of the following:
 - a) In the bada IDE, click the Terminate button in the Console or Debug view.
 - b) In the target device, press the End key.
8. After the run, the results of the application profiling are displayed in the Profile view.
9. Disconnect the target device.

While running the application, you can also use the Performance Analyzer tool to check your application's performance during execution.

Using a Target Device in Debugging

The target device debugging has the following limitations:

- ▶ Due to hardware limitations, you can insert at most 10 breakpoints.
- ▶ You cannot insert watchpoints to monitor the variables used in your application.
- ▶ You cannot use the Step Into option (a button in the Debug view) for target debugging; in the debugging mode, bada applications are built using the `-fpicGDB` command, which means that the addresses of the symbols, such as variables and APIs, cannot be determined unless they are loaded in the memory. To examine the application as it steps into disassembled code, enable the Instruction Stepping Mode option in the Debug view.
- ▶ Do not step out during debugging inside the `OspMain()` event handler